# Tema 4: Interfaces Gráficas con Swing
# Menus en Java

Pablo Gordillo (pabgordi@ucm.es)

*Departamento de Sistemas Informáticos y Computación*

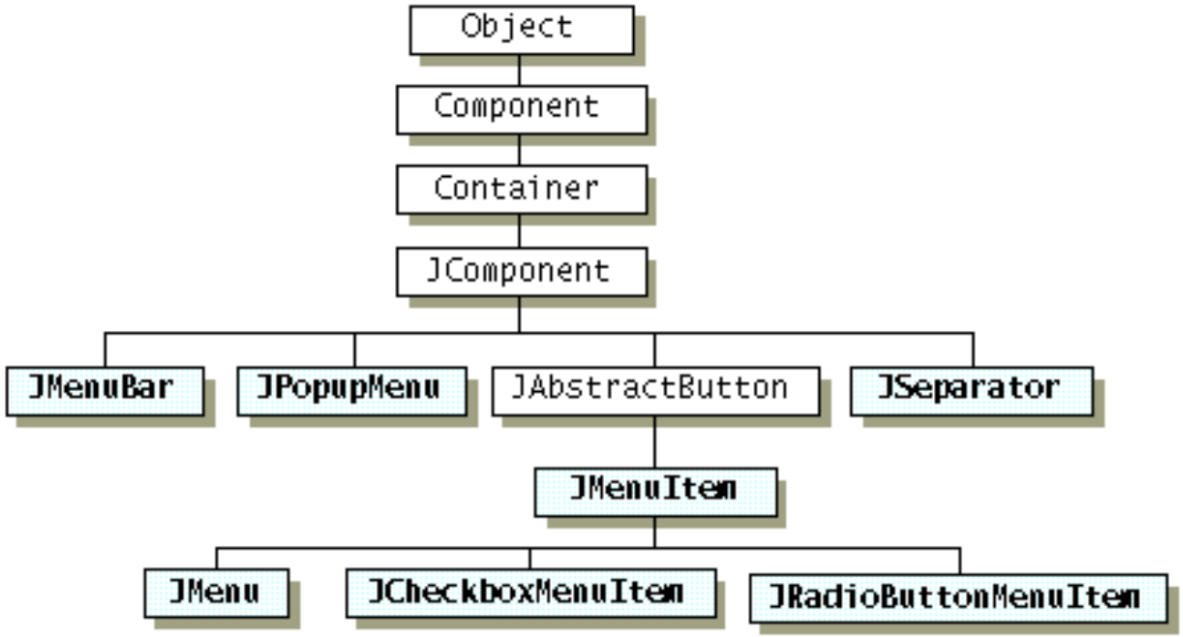
Facultad de Informática




(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

# Menús en Swing- JMenuBar

- A diferencia del resto de componentes, un menú **se coloca en la barra de menú de la ventana o como un meú emergente (pop-up)**
- Un menú emergente (pop-up) no es visible hasta que el usuario lo hace visible a través de alguna opción (como presionar el botón derecho del ratón).



# Crear una barra de menú

```
JMenuBar menuBar = new JMenuBar();
frame.setJMenuBar(menuBar);
```

Una vez creada la barra de menú sobre el frame, podemos añadirle menús. Primero se crea el menú, y después se añade a la barra de menús.

```
JMenu menu = new JMenu(String);
menuBar.add(menu);
```

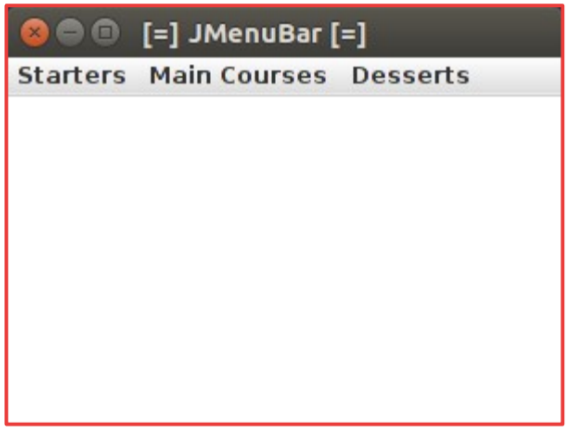
Una vez creados los menús, podemos añadirles items, radioButtons, o checkbox.

```
JMenuItem item = new JMenuItem(String);
menu.add(item);
JRadioButtonMenuItem rButton = new JRadioButtonMenuItem(String);
menu.add(rButton);
JCheckBoxMenuItem checkBox = new JCheckBoxMenuItem(String);
menu.add(checkBox);
```

# Crear una barra de menú: ejemplo

```
public  JMenuBar createMenuBar() {
   JMenuBar menuBar = new JMenuBar();
   JMenu starter = new JMenu("Starters");
   JMenu maincourse = new JMenu("Main Courses");
   JMenu desserts = new JMenu("Desserts");
   menuBar.add(starter);
   menuBar.add(maincourse);
   menuBar.add(desserts);
   ...

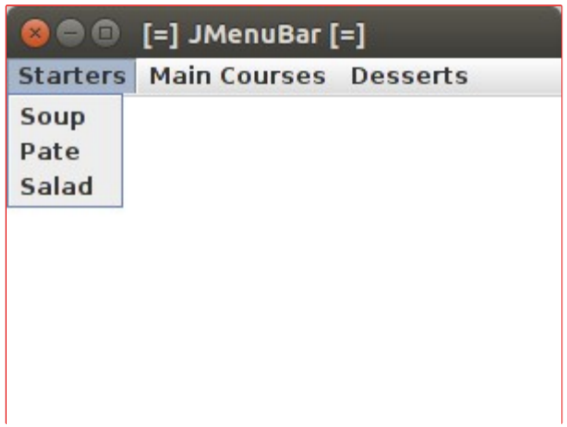
   // código para el menú "Starters"
   // Código para el menú "Main Courses"
   // Código para el menú "Desserts"
   ...
   return  menuBar;
}
```



# Crear una barra de menú: ejemplo

```
JMenuItem soup = new JMenuItem("Soup");
JMenuItem pate = new JMenuItem("Pate");
JMenuItem salad = new JMenuItem("Salad");

starter .add(soup);
starter .add(pate);
starter .add(salad);
```



# Crear una barra de menú: ejemplo

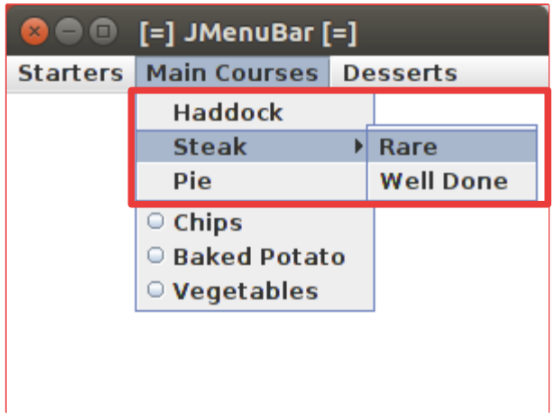
```
JMenuItem fish = new JMenuItem("Haddock");
maincourse.add(fish );

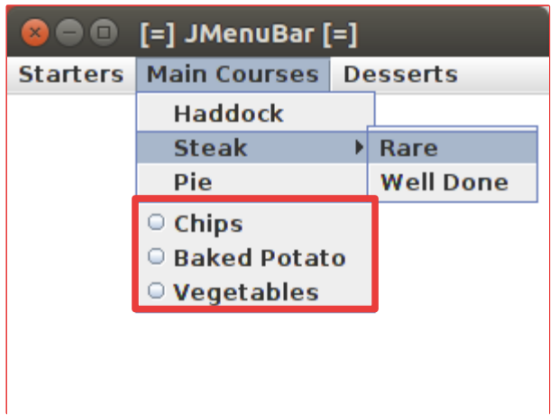
JMenu steak = new JMenu("Steak");
JMenuItem rare = new JMenuItem("Rare");
JMenuItem welldone = new JMenuItem("Well Done");
steak.add(rare);
steak.add(welldone);
maincourse.add(steak);

JMenuItem pie = new JMenuItem("Pie");
maincourse.add(pie);

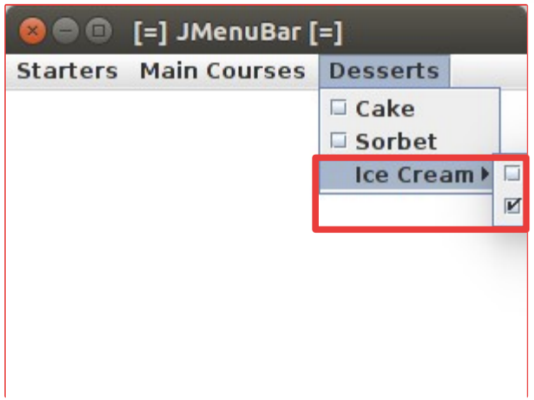
maincourse.addSeparator();

JRadioButtonMenuItem chips = new JRadioButtonMenuItem("Chips");
JRadioButtonMenuItem bp = new JRadioButtonMenuItem("Baked Potato");
JRadioButtonMenuItem vege = new JRadioButtonMenuItem("Vegetables");
ButtonGroup sides = new ButtonGroup();
sides.add(chips);
sides.add(bp);
sides.add(vege);
maincourse.add(chips);
maincourse.add(bp);
maincourse.add(vege);
```



# Crear una barra de menú: ejemplo



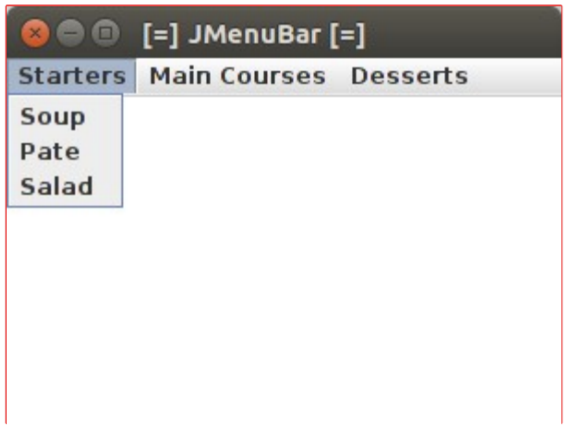

```
JCheckBoxMenuItem cake = new JcheckBoxMenuItem("
desserts . add(cake);
JCheckBoxMenuItem sorbet = new JcheckBoxMenuItem(
desserts . add(sorbet);
JMenu icecream = new JMenu("Ice Cream");
JCheckBoxMenuItem choc = new JCheckBoxMenuItem("Chocolate");
JCheckBoxMenuItem vani = new JCheckBoxMenuItem("Vanilla");
icecream. add(choc);
icecream. add(vani);
desserts . add(icecream);
```

# Menús en Swing - Listeners

- Cuando un usuario elige un **MenuItem**, el evento generado es una acción, y por lo tanto debemos utilizar **ActionListener**. Lo mismo ocurre en general cuando se selecciona un **RadioButtonMenuItem**. Para el caso de un **CheckBoxMenuItem**, lo habitual es tratar el evento "item" que se lanza.

```
    JMenuItem soup = new JMenuItem("Soup");
    soup.addActionListener(this);
    JMenuItem pate = new JMenuItem("Pate");
    pate.addActionListener(this);
    JMenuItem salad = new JMenuItem("Salad");
    salad.addActionListener(this);
    ...
}

  public void actionPerformed(ActionEvent e) {
    System.out.println(e.getActionCommand() + " Selected");
  }
```

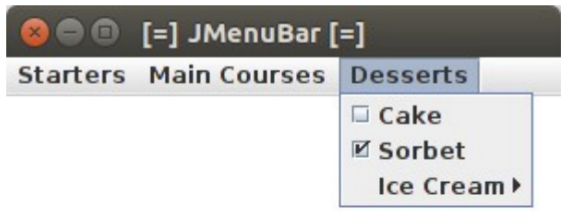


# Menús en Swing - Listeners

- Cuando un usuario elige un **MenuItem**, el evento generado es una acción, y por lo tanto debemos utilizar **ActionListener**. Lo mismo ocurre en general cuando se selecciona un **RadioButtonMenuItem**. Para el caso de un **CheckBoxMenuItem**, lo habitual es tratar el evento "item" que se lanza.

```
    cake = new JCheckBoxMenuItem("Cake");
    cake.addItemListener(this);
    sorbet = new JCheckBoxMenuItem("Sorbet");
    sorbet.addItemListener(this);
    ...
}

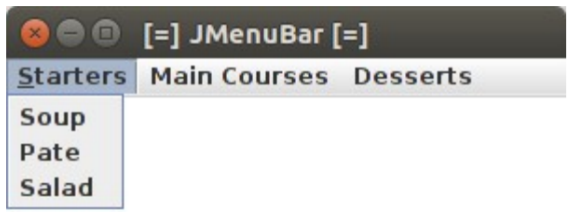
public void itemStateChanged(ItemEvent e) {
    if (e.getSource() == cake) System.out.println("Cake Clicked");
    else if (e.getSource() == sorbet) System.out.println("Sorbet Clicked");
}
```



# Menús en Swing - Teclas nemotécnicas

- Permiten navegar por el menú utilizando el teclado.
- Gráficamente se muestran con un subrayado sobre el carácter que hace de nemotécnico. Si dicho carácter no aparece en el nombre, entonces no se visualiza gráficamente nada.
- En la clase **KeyEvent** aparecen todas las constantes estáticas que se pueden utilizar como teclas nemotécnicas.

```
JMenu starter = new JMenu("Starters");
starter .setMnemonic(KeyEvent.VK_S);
```

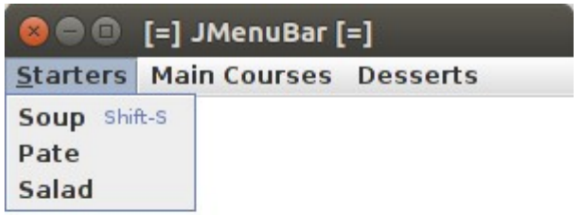



**Al teclear ATL + S se abre el menú para Starters.**

# Menús en Swing - Teclas aceleradoras

- Permiten introducir "shorcuts" para hacer referencia a un menú.
- Son visibles en el item correspondiente del menú.
- Permiten fijar con que tecla de control ejecutar (CRTL, ALT, SHIFT,etc).

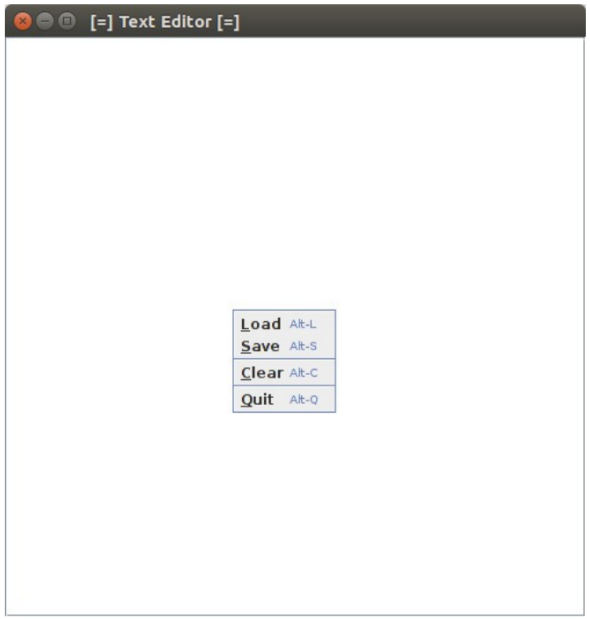
```
JMenuItem soup = new JMenuItem("Soup");
soup.setAccelerator (KeyStroke.getKeyStroke(KeyEvent.VK_S,
                     ActionEvent.SHIFT_MASK));
```




**Al pulsar Mayúsculas + S, se ejecuta la acción asociada a "Soup".**

# Colocando un Pop-up a una componente

- Un **Pop-up** menú es un menú emergente que puede aparecer al pulsar el ratón, o una letra, etc.
- Se construyen igual que los menús normales, pero se colocar sobre componentes y no sobre la barra de menú.




**En este caso el menú emergente sale cuando pulsamos el ratón.**

# Colocando un Pop-up a una componente

```
this.popup = createPopUpMenu(); // se crea el pop-up menu

textArea = new JTextArea("");
textArea.setEditable (true);  // definimos la componente sobre la
textArea.setLineWrap(true);  // que saldrá el pop-up
textArea.setWrapStyleWord(true);
textArea.addMouseListener(new MouseListener(){ // añadimos MouseListener
                                                // al area de texto

    @Override
    public void mouseClicked(MouseEvent e) {
      popup.show(e.getComponent(), e.getX(), e.getY());
      // lo muestra donde se pulse
    }
    @Override
    public void mouseEntered(MouseEvent arg0) {}
    @Override
    public void mouseExited(MouseEvent arg0) {}
    ...
});
```